RAMACHANDRA COLLEGE OF ENGINEERING

(Autonomous)

COURSE:OBJECT-ORIENTED PROGRAMMING THROUGH JAVA

Unit-03: ARRAYS & STRING HANDLING IN JAVA

Unit-III:

Arrays: Introduction, Declaration and Initialization of Arrays, Storage of Array in Computer Memory, Sorting of Arrays, Search for Values in Arrays, Two-dimensional Arrays, Arrays of Varying Lengths, Three-dimensional Arrays.

String Handling in Java: Introduction, Interface Char Sequence, Class String, Methods for Extracting Characters from Strings, Comparison, Modifying, Searching, Class String Buffer, Class String Builder.

- ✓ **The complete Reference Java, 11th Edition, Herbert Schildt, TMH**

- ✓ **Joy with JAVA, Fundamentals of Object-Oriented Programming, Debasis Samanta, Monalisa Sarma, Cambridge, 2023.**

LEARNING OBJECTIVE

Make the student able to Understanding to the concepts of arrays and handling strings in Java to manipulate data effectively

LEARNING OUTCOMES

After attending this session a student will be able to use the concepts of arrays and handling strings in Java to manipulate data effectively.

Part-A:ARRAYS

Concept-1:

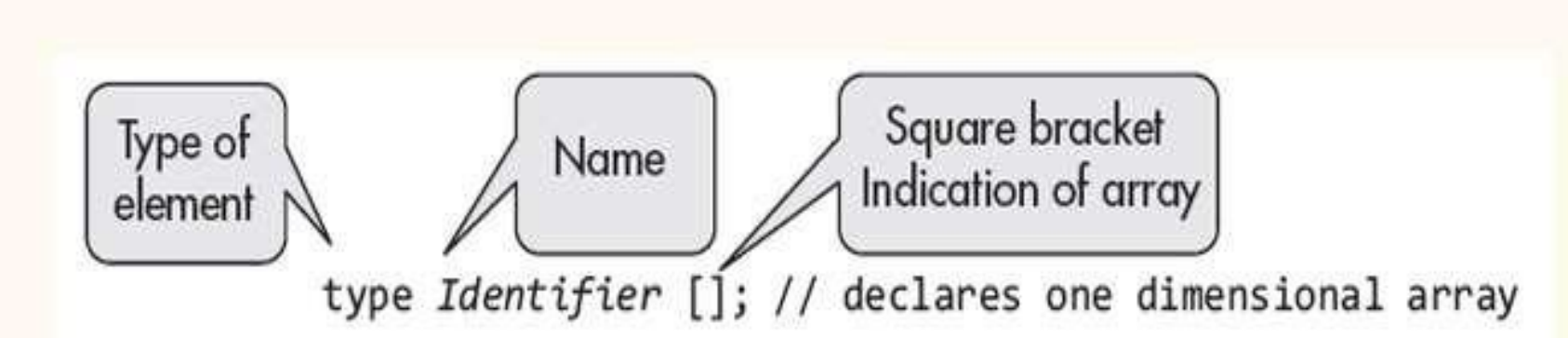
Introduction, Declaration and Initialization of Arrays

- **Definition**: An array is a structure that consists of a group of elements, all of which are of the same type. This allows for efficient processing when dealing with large numbers of data values of the same type.
- **Memory Representation**: In computer memory, the complete data set of the array is represented by a single object, identified by a single name.
- **Data Type:** The type of data that the array holds determines the type of the array, also known as the base type. For example:
- If the array contains whole numbers, the type of the array is int
- If the array contains characters, the type of the array is char
- If the array contains floating-point numbers, the type of the array is float.
- **Object Storage:**An array can hold objects of a class, but it cannot hold a mixture of different data types

Syntax: **datatype arrayName[];**

Or

type identifier[];



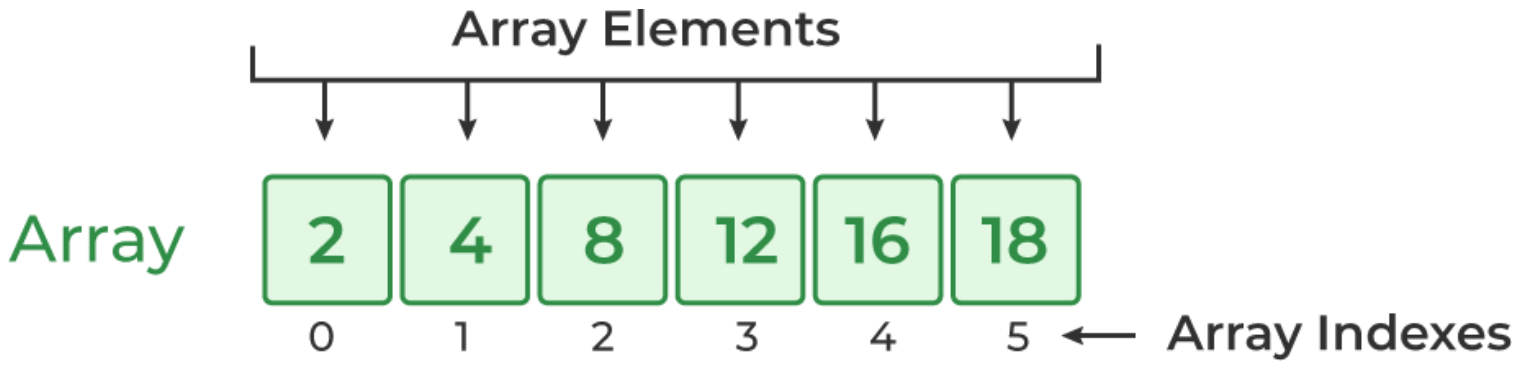
Examples: **int numbers[ ]; // an array of whole numbers**

char name[ ]; // A name is an array of characters

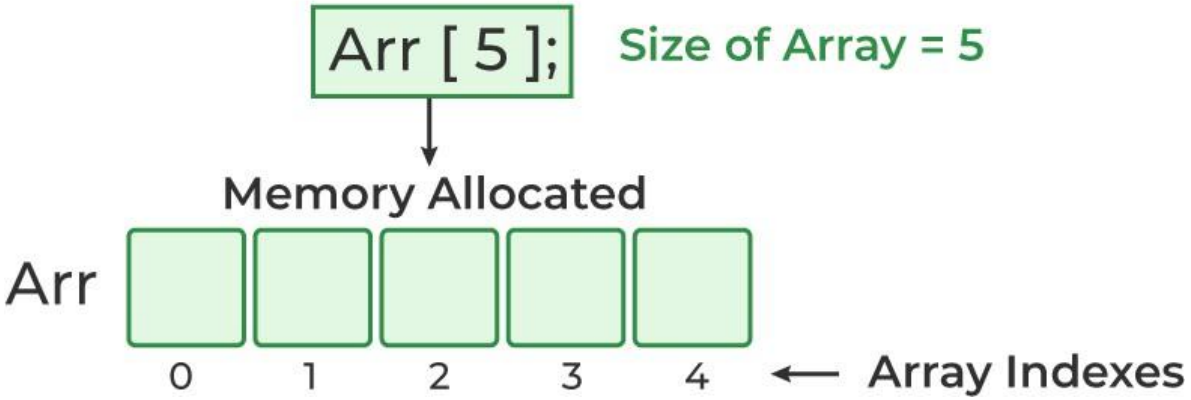
float priceList[ ]; // An array of floating point numbers.

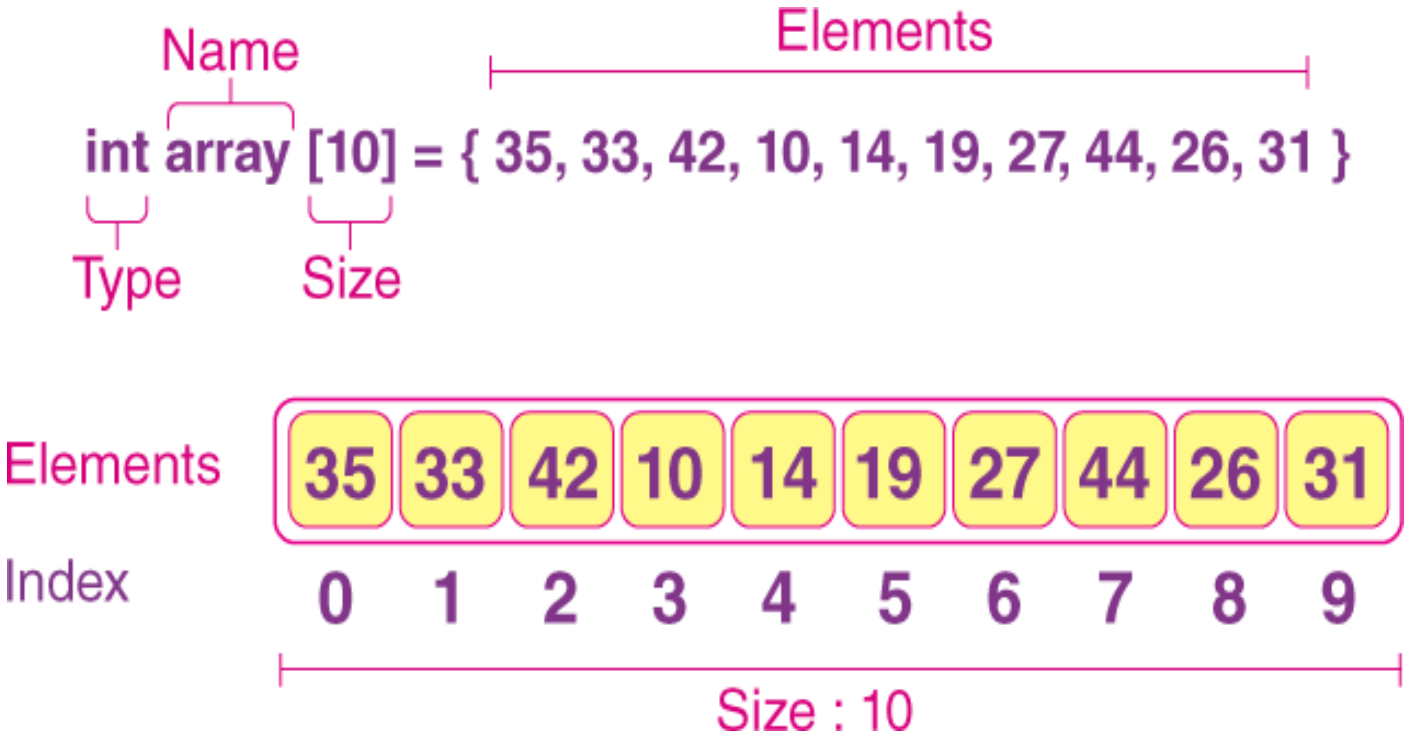
- ✓ **Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.**
- ✓ **Group of elements with same data type is called as Array**
- ✓ **To create an array, define the data type (like int,char,float) and specify the name of the array followed by square brackets [].**
- ✓ **To insert values to it, use a comma-separated list, inside curly braces:**

```
int myNumbers[] = {2,4,8,12,16,18};
```



Array Declaration




Name
Elements
int array [10] = { 35, 33, 42, 10, 14, 19, 27, 44, 26, 31 }
Type
Size
Elements
35 33 42 10 14 19 27 44 26 31
Index
0 1 2 3 4 5 6 7 8 9
Size : 10

Advantage of Array in java

1) Code Optimization: Less code to the access the data.

2) Ease of traversing: By using the for loop, we can retrieve the elements of an array easily.

3) Ease of sorting: To sort the elements of the array, we need a few lines of code only.

4) Random Access: We can access any element randomly using the array.

Disadvantage of Array in java

Fixed Size: Whatever size, we define at the time of declaration of the array, we can't exceed the limit.

Properties of Array

The array contains the following properties.

- ✓ Each element of an array is of same data type and carries the same size, i.e., int = 4 bytes.
- ✓ Elements of the array are stored at continuous memory locations where the first element is stored at the smallest memory location.
- ✓ Elements of the array can be randomly accessed since we can calculate the address of each element of the array with the given base address and the size of the data element.

Arrays in Java – Initialization and Memory Representation

1 Step 1: Declaration

```
int[] x;
```

x is the name of array list.



Next →

2 Step 2: Memory Allocation

```
x = new int[6];
```

Memory is allocated for 6 integer elements.



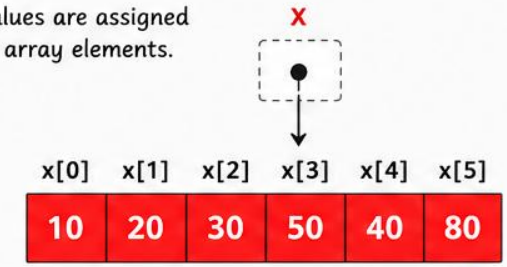


Next →

3 Step 3: Initialization

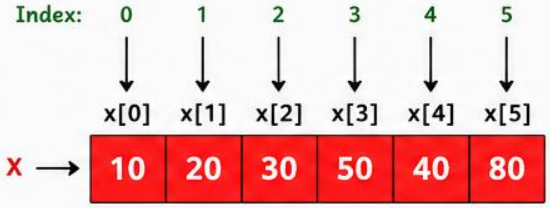
```
x = {10, 20, 30, 50, 40, 80};
```

Values are assigned to array elements.



Next →

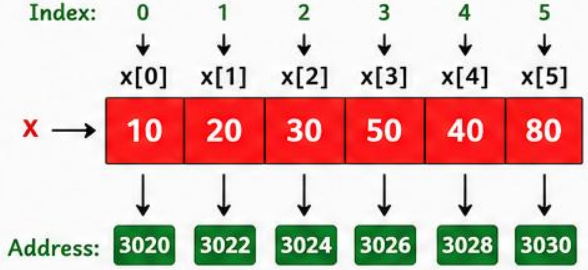
4 Step 4: Indexes



Each element is accessed using its index.

Next →

5 Step 5: Memory Addresses



(Assume each int occupies 2 bytes)

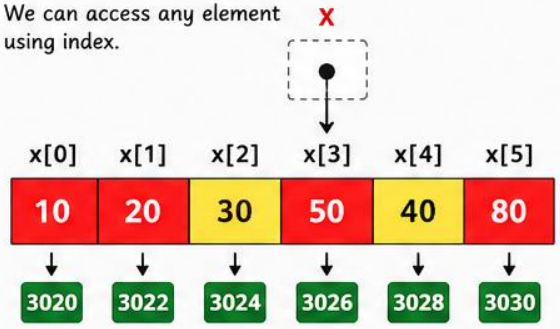
- Address of x[0] is 3020
- Each next element is stored in next memory location.

Next →

6 Step 6: Accessing Elements

```
x[2] = 30;    // 3rd element
x[4] = 40;    // 5th element
```

We can access any element using index.



• Arrays store similar type of elements in **contiguous** memory locations. Elements are accessed using **indexes** starting from 0.

Part-A:ARRAYS

Concept-2:

Storage of Array in Computer Memory

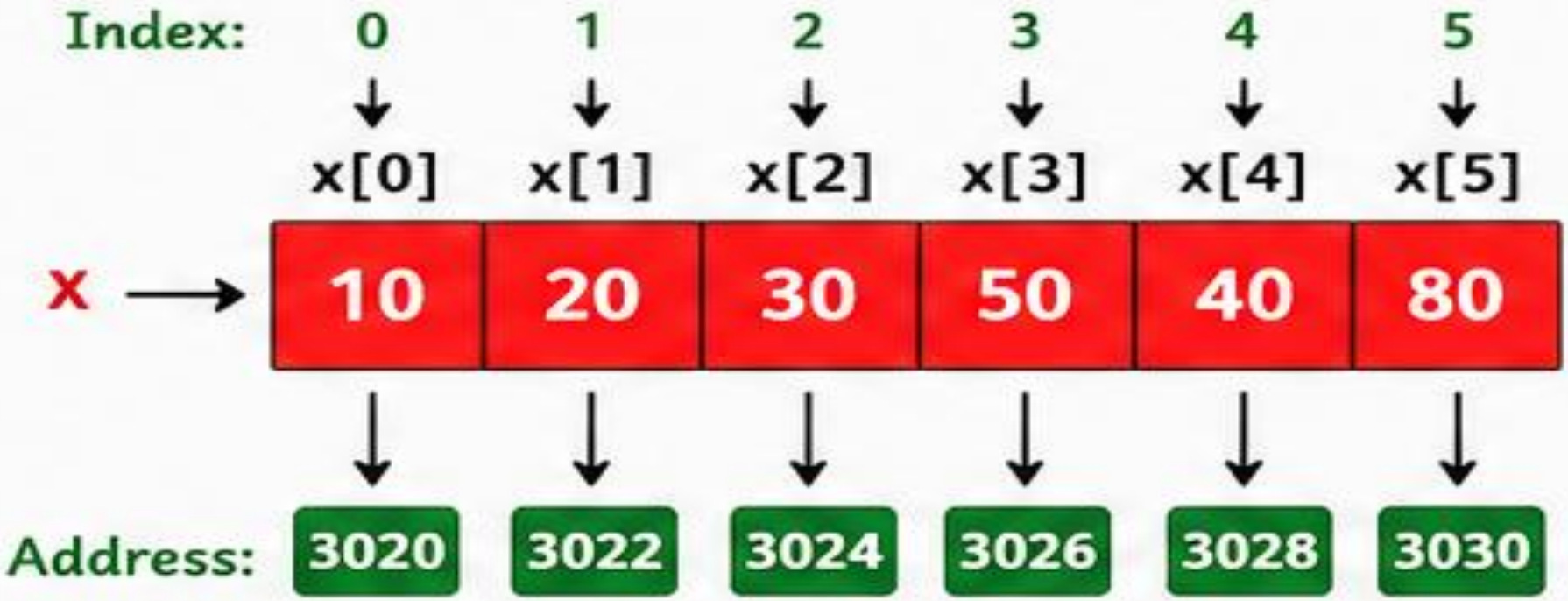
- ✓ **The operator new allocates memory for storing the array elements.**
- ✓ **After array declaration no memory is allocated. Memory is allocated when the operator new is used or when it is initialized along with a declaration.**

datatype identifier[]=new datatype[size];

Example: **int numbers[] = new int[6];**

- ✓ **Here 6 elements will be created and assigned to the array "numbers"**
- ✓ **When an array is created as above, element of the array is automatically initialized to 0 by the compiler.**
- ✓ **After allocation of memory as above, each member may be assigned a value by accessing it by its index value.**
- ✓ **number[0]=10; number[1]=20; number[2]=30;number[3]=50 etc;**
- ✓ **The declaration and initialization may as well be combined as:**

int numbers [] = {10,20,30,50,40,80};



(Assume each int occupies 2 bytes)

- Address of x[0] is 3020
- Each next element is stored in next memory location.

Part-A:ARRAYS

Concept-3:

Accessing Elements of Arrays

- ✓ The individual member of an array may be accessed by its index value.
- ✓ The index value represents the place of element in the array.
- ✓ The first element space is represented by numbers [0], and index value is 0.
- ✓ Note that the value of an array element is different from its index value.

EXAMPLE-1

```
class ArrayDemo
{
public static void main(String args[])
{
int a[ ]=new int[3];
a[0]=10;
a[1]=20;
a[2]=30;
for(int i=0;i<a.length;i++)
{
System.out.println(a[i]);
}
}
}
```

Output:

```
E:\java>javac ArrayDemo.java

E:\java>java ArrayDemo
10
20
30
```

We can declare, instantiate and initialize the java array together by:

EXAMPLE-2

```
class ArrayDemo
{
public static void main(String args[])
{
int a[]={10,20,30};
for(int i=0;i<a.length;i++)
{
System.out.println(a[i]);
}
}
}
```

OUTPUT:

```
E:\java>javac array1.java

E:\java>java Array
10
20
30
```

EXAMPLE-3

Java Program to print the array elements using for-each loop

```
class Array
{
public static void main(String args[])
{
int arr[]={10,20,30};
for(int i:arr)
{
System.out.println(i);
}
}
}
```

OUTPUT

```
E:\java>javac array2.java

E:\java>java Array
10
20
30
```

```
class StringArray

{

public static void main(String args[])

{

String names[] = {"Red", "Blue", "Green", "Black", "White"} ;

for(String i :names)

{

System.out.println(i);

}

}

}
```

```
C:\javap>javac StringArray.java

C:\javap>java StringArray
Red
Blue
Green
Black
White
```

EXAMPLE-4

```
import java.util.Scanner;
class Arr
 {
    public static void main(String[] args)
 {
  Scanner sc = new Scanner(System.in);
  int a[ ] = new int[5];   // Declare and initialize the array
System.out.println("Enter 5 integers:"); // Read user input for the array elements
        for (int i = 0; i < a.length; i++)
 {
            a[i] = sc.nextInt();
        }

System.out.println("Array elements are:");
        for (int i = 0; i < a.length; i++) // Print the array elements
{
            System.out.println("Element at index " + i + ": " + a[i]);
        }

    }
}
```

Part-A:ARRAYS

Concept-4:

Assigning Array to Another Array

- **In Java, one array may be assigned to another array of same data type.**
- **In Java, it will copy all the elements of one array into another array.**
- **In this process, the second array name contains the address is assigned to the first array.**
- **The second array is not a new array, it contain the reference of the first array.**
- **array1 is assigned to array2. Then, array2 is modified array1 also gets modified, array1 is not show an independent array.**

```
class AssignArray
{
public static void main(String args[ ])
{
Int array1[] = {1, 2, 3, 4, 5};
Int array2[] = array1; //Assigning Array to Another Array
for(int i=0;i<array1.length;i++) //first array
{
System.out.println(array1[i]);
}
for(int j=0;j<array2.length;j++) //second array
{
System.out.println(array2[j]);
}
}
}
```

```
C:\javap>javac AssignArray.java

C:\javap>java AssignArray
1
2
3
4
5
1
2
3
4
5
```

```
class AssignArray
{
public static void main(String args[ ])
{
int array1 [] = {1, 2, 3, 4, 5};
int array2 [] = array1; ;//Assigning Array to Another Array
array2[0]=10;
array2[1]=20;
for(int i=0;i<array1.length;i++) //first array
{
System.out.println(array1[i]);
}
for(int j=0;j<array2.length;j++) //second array
{
System.out.println(array2[j]);
}
}
}
```

```
C:\javap>javac AssignArray.java

C:\javap>java AssignArray
10
20
3
4
5
10
20
3
4
5
```

```
class AssignArray
{
public static void main(String args[ ])
{
int array1[] = {1, 2, 3, 4, 5};
int array2[] = new int[array1.length];
for(int i=0;i<array1.length;i++) //first array
{
array2[i]=array1;
}
for(int j=0;j<array2.length;j++) //Second array
{
System.out.println(array1[j]);
}
}
}
```

```
C:\javap>javac AssignArray.java

C:\javap>java AssignArray
1
2
3
4
5
```

These arrays are having dimensions

I. Single Dimensional Array
II. Multi Dimensional Array (more than one)
a. Two Dimensional Array
b. Jagged Array

Note

✓ **example:**

:Dimension are represented by square brackets or subscripts []

✓ marks[]

✓ marks[][]

✓ marks[][][]

Part-A:ARRAYS

Concept-5:

Two-dimensional Arrays

- **A two dimensional array is declared as follows:**

  datatype identifier[][];

For Example:- **int number[][];**

- **The number of square brackets indicates the dimension or depth of an array.**
- **The Memory allocation may be achieved by the operator new as:**

Datatype arrayname[][]=new datatype[rowssize][columnsize]

For Example:-

int number[][]=new int[3][2]

- **In the above Example rows size is 3 and column size is 2 hence 3x2 Matrices is created.**

Alternative array Declaration syntax:-

- Use of a square bracket before and after the identifier will create a multi dimensional array.

```
int[] number[];
```

- The above declaration is equivalent to

```
int[][] number;      Or      int  number[][];
```

Declaration and initialization of array:-

- A two dimensional array may be declared and initialized as

```
int number[][] =new int[][] {{1,2,3}, { 4,5,6}}
```

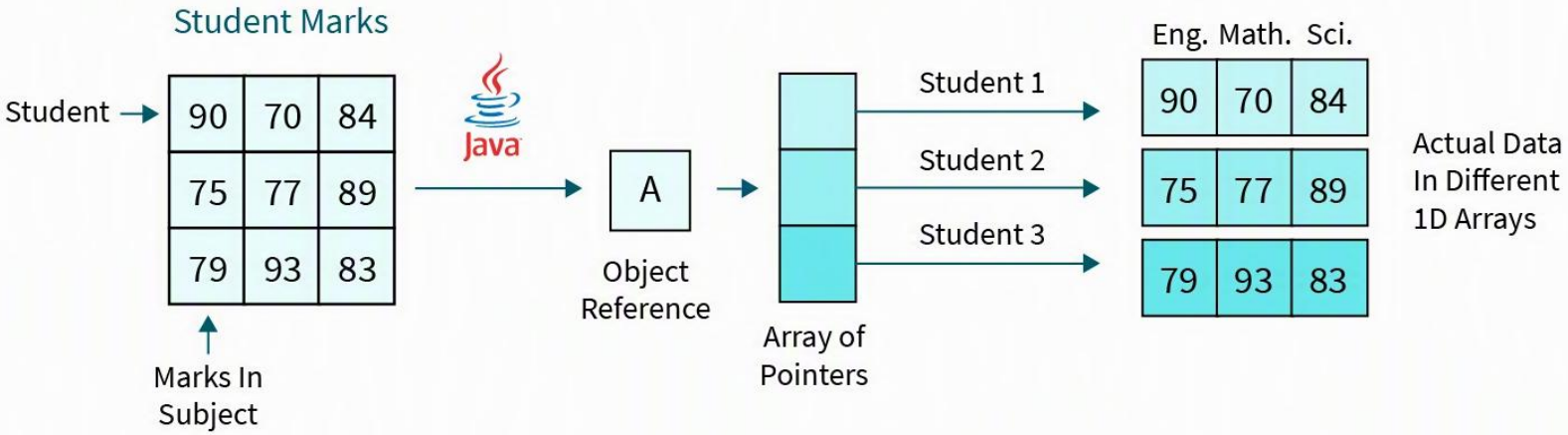
Or

```
int [][] number={{1,2,3},{4,5,6}}
```

- **A two-dimensional array is treated as an array of arrays, and each of these arrays may have a different number of elements.**
- **<u>For Example:-</u> Matrices is the example for two-dimensional arrays.**
- **Array index start from 0th index not 1st index. And 2D array also start from 0th row and 0th column index.**

	Column 1	Column 2	Column 3	Column 4
Row 1	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 2	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 3	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Java's Two Dimensional Arrays Example -



```
class TwoDimenArray
{
    public static void main(String args[])
{
        // Initialize a 3x3 matrix
        int a[][] = {
            {1, 1, 1},
            {2, 2, 2},
            {3, 3, 3}
        };

        // Loop through rows of the matrix
 for (int i = 0; i < 3; i++)
         {
            for (int j = 0; j < 3; j++) // Loop through columns of the matrix
            {
              System.out.print(a[i][j] + " ");   // Print each element in the row
            }
          System.out.println();   // Move to the next line after printing all columns in the row
        }
    }
}
```

```
C:\javap>javac TwoDimenArray.java

C:\javap>java TwoDimenArray
1 1 1
2 2 2
3 3 3
```

```
class TwoDimArray
{
    public static void main(String args[])
 {
        // Initialize a 2D array with predefined values
        int num2D[][] = {{1, 2, 3},{4, 5, 6},{7, 8, 9}};
        // Enhanced for loop to iterate over the 2D array
        for (int[] row : num2D) // Iterate over each row in the array
         {
            for (int element : row)  // Iterate over each element in the current row
         {
           System.out.print(element + " ");   // Print the element followed by a space
            }
            System.out.println();  // Print a newline after each row
        }
    }
}
```

```
C:\javap>javac TwoDimArray.java

C:\javap>java TwoDimArray
1 2 3
4 5 6
7 8 9
```

```
class MatrixMultiplication
 {
    public static void main(String args[])
 {
        // Initialize two matrices
        int a[][] = {{1,1,1},{2,2,2},{3,3,3}};
        int b[][] = {{1,1,1},{2,2,2},{3,3,3}};

        // Initialize the result matrix with zeros
        int c[][] = new int[3][3];
// Perform matrix multiplication
```

```
for (int i = 0; i < 3; i++) // Loop through rows of matrix a
{
        for (int j = 0; j < 3; j++) // Loop through columns of matrix b
        {
        c[i][j] = 0;  // Initialize the result matrix element to zero
                for (int k = 0; k < 3; k++)
                {
          // Loop through elements for multiplication
                c[i][j] += a[i][k] * b[k][j];
                }
        System.out.print(c[i][j] + " ");
       }
System.out.println();  // Move to the next line after each row
}
}       }
```

```
C:\javap>javac MatrixMultiplication.java

C:\javap>java MatrixMultiplication
6 6 6
12 12 12
18 18 18
```

Part-A:ARRAYS

Concept-6:

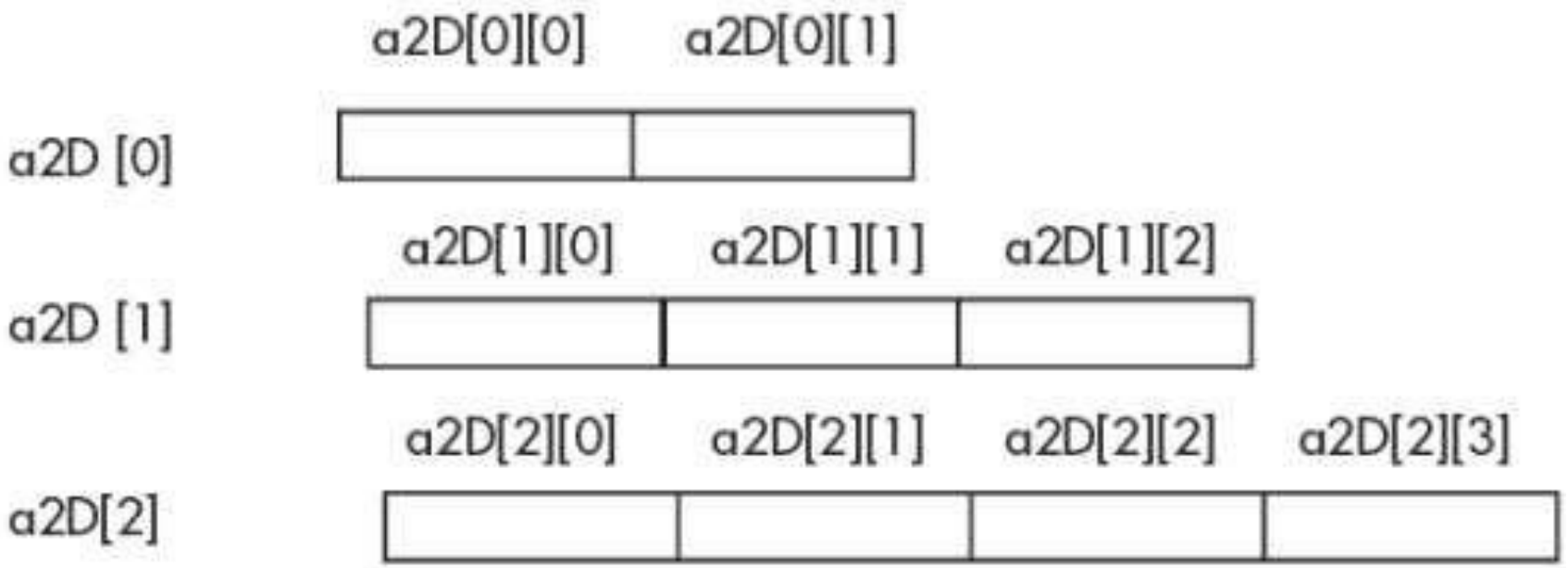
Arrays of Varying Lengths

Or

Jagged array

- **A jagged array is an array of arrays such that member arrays can be of different sizes, i.e., we can create a 2-D array but with a variable number of columns in each row.**
- **These types of arrays are also known as Jagged arrays or non-rectangular arrays.**
- **You have to first declare and instantiate the array by specifying only the number of elements in the first dimension and leaving the second dimension empty.**

  For example: `int table[][] = new int[3][];`
- **The arrays may as well be declared as**
- `int array [][] ={{5,7,8},{10,11} {4,3,2,7,5}};`


a2D[0][0]
a2D[0][1]
a2D [0]
a2D[1][0]
a2D[1][1]
a2D[1][2]
a2D [1]
a2D[2][0]
a2D[2][1]
a2D[2][2]
a2D[2][3]
a2D[2]

```
class JaggedArray
{
    public static void main(String args[])
 {
        // Initialize a jagged array with different row lengths

        int a[][] = {{1,1},{2,2,2},{3,3,3,3}};
        // Loop through each row of the jagged array
        for (int i = 0; i < a.length; i++)
{
            // Loop through each column in the current row (length may vary)
            for (int j = 0; j < a[i].length; j++)
{
                System.out.print(a[i][j] + " ");   // Print each element in the row
            }
            System.out.println();  // Move to the next line after printing all
elements in the row
        }
    }
}
```

```
C:\javap>javac JaggedArray.java

C:\javap>java JaggedArray
1 1
2 2 2
3 3 3 3
```

Part-A:ARRAYS

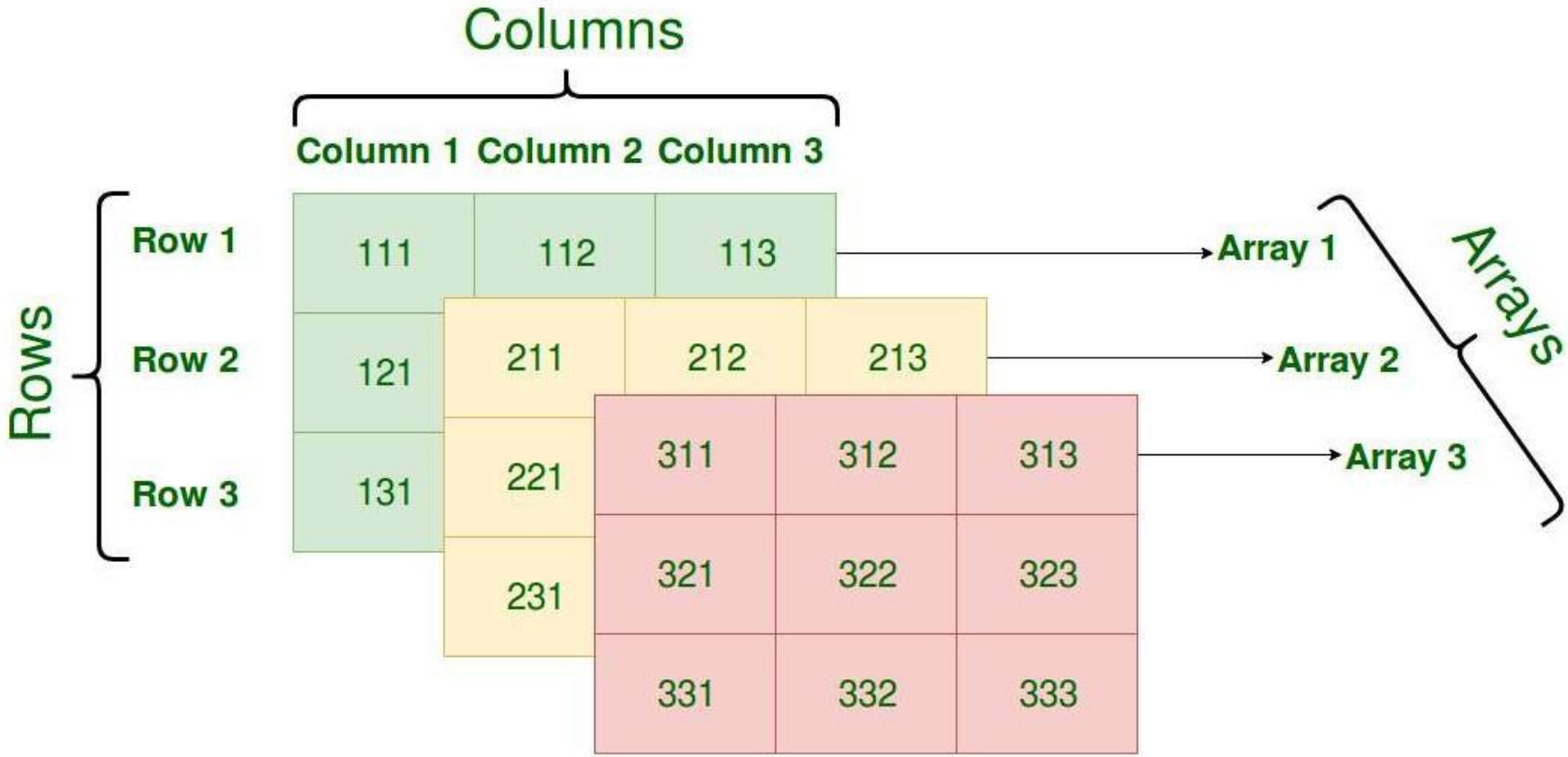
Concept-7:

Three-dimensional Arrays

- ✓ **Three-dimensional array is the collection of two-dimensional arrays in Java programming language.**
- ✓ **Three-dimensional array is also called the multidimensional array.**
- ✓ **Each basic element of such an array needs three index values for its reference.**

Syntax: **datatype identifier[][][]=new datatype[][][];**

Example: **int array[][][]=new int[2][3][3];**


Columns
Column 1
Column 2
Column 3
Rows
Row 1
Row 2
Row 3
111
112
113
121
211
212
213
131
221
311
312
313
231
321
322
323
331
332
333
Array 1
Array 2
Array 3
Arrays

```
class ThreeDimenArray {
    public static void main(String args[]) {
        // Initialize a jagged 3x2x2 3D array
        int a[][][] = {{{1,1},{2,2}},{{3,3},{4,4}},{{5,5},{6,6}}};
        // Loop through each "layer" of the 3D array
   for (int i = 0; i < a.length; i++) // Loop through each 2D array (layer)
{
System.out.println("Layer " + i + ":");
   for (int j = 0; j < a[i].length; j++) // Loop through each row in the 2D array
{
for (int k = 0; k < a[i][j].length; k++) // Loop through each column in the row
{
   System.out.print(a[i][j][k] + " "); // Print each element
   }System.out.println(); // Move to the next line after printing all columns in the row
  }
 System.out.println(); // Separate layers with a new line
        }        }        }
```

```
C:\javap>javac ThreeDimenArray.java

C:\javap>java ThreeDimenArray
Layer 0:
1 1
2 2

Layer 1:
3 3
4 4

Layer 2:
5 5
6 6
```

Part-A:ARRAYS

Concept-8:

Search for Values in Arrays

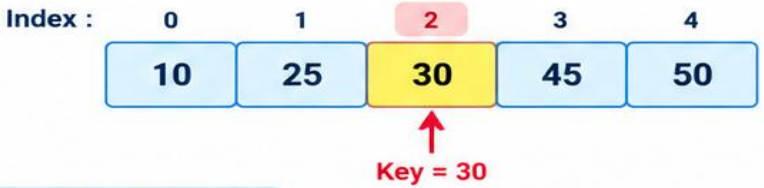
Searching in data-structure refers to the process of finding a desired element in set of items.

Linear Search

Linear Search checks each element of the array **one by one** from the **beginning** until the required element is found or the end of the array is reached.

Example

Find the element **30** in the array.



Step-by-Step Process

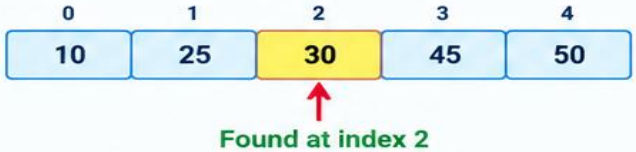
1 Compare 10 with 30 → Not equal



2 Compare 25 with 30 → Not equal



3 Compare 30 with 30 → Equal (Found!)



Key Points - Linear Search

- Checks elements **one by one**.
- Works on sorted or unsorted arrays.
- Simple to implement.
- Time Complexity: **O(n)**

Binary Search

Binary Search works on a **sorted array**. It repeatedly divides the search range into two halves and continues the search in the appropriate half.

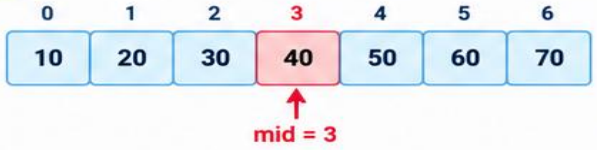
Example

Find the element **50** in the sorted array.



Step-by-Step Process

1 Initial: low = 0, high = 6, mid = (0+6)/2 = 3



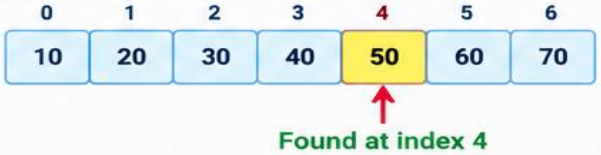
40 < 50
Search right half
(low = 4)

2 Now: low = 4, high = 6, mid = (4+6)/2 = 5



60 > 50
Search left half
(high = 4)

3 Now: low = 4, high = 4, mid = (4+4)/2 = 4



Key Points - Binary Search

- Divides the array into two halves.
- Works only on sorted arrays.
- Much faster for large arrays.
- **Time Complexity: O(log n)**

```
import java.util.Scanner;
class LinearSearch
{
    public static void main(String args[])
{
        boolean b = false; // Flag to indicate if the key is found
        int array[] = {67, 78, 85, 44, 25, 65, 36}; // Array to search sc
        Scanner sc = new Scanner(System.in); // Scanner to take input
        // Prompt user to enter the number they want to search
        System.out.println("Enter the number which you want to search: ");
        int key = sc.nextInt(); // Read the number to search
```

```
for (int i = 0; i < array.length; i++) // Linear search to find the key in the array
 {
            if (array[i] == key)
{
                System.out.println("Your number is at index = " + i);
                b = true;  // Set the flag to true if the key is found

                break;  // Exit the loop once the key is found
            }
        }

        if (!b) // If the key is not found, print a message
{
            System.out.println("Your number is not in the array");
        }
    }
}
```

```
C:\javap>javac LinearSearch.java

C:\javap>java LinearSearch
Enter the number which you want to search:
11
Your number is not in the array

C:\javap>java LinearSearch
Enter the number which you want to search:
65
Your number is at index = 5
```

Part-A:ARRAYS

Concept-9:

Dynamic Change of Array Size

- **Java allows us to change the array size dynamically during the execution of the program.**
- **In C and C++ where the array is once declared it size fixed, that is, the number of elements cannot be changed.**
- **We may change the number of elements by dynamically but we can retain the name of array.**
- **In this process the old array is destroyed along with the values. After change the array size the element are destroyed and that array is default initialized to the 0.**

```
class DyanamicArraySize

{

public static void main(String args[])

{

int array[]= {1,2,3,4,5}; //declare array

System.out.println("Before Changing Array Size: array1 = ");

display(array); //display array

array = new int[10]; //Changing array size

System.out.println("\nAfter Changing Array Size: array1 = ");

display(array);
```

```
for(int i=0;i<10;i++) //adding values to the arrayelements
 {
array[i] =(i+1);
}
System.out.println("\nAfter Modification : array = ");
display(array);
}
static void display(int array[])  //Method definition
{
for(int i=0;i<array.length;i++)
{
System.out.print(array[i]+ "   ");
}
}
}
```

```
C:\javap>javac DyanamicArraySize.java

C:\javap>java DyanamicArraySize
Before Changing Array Size: array1 =
1   2   3   4   5
After Changing Array Size: array1 =
0   0   0   0   0   0   0   0   0   0
After Modification : array =
1   2   3   4   5   6   7   8   9   10
```

Part-A:ARRAYS

Concept-10:

Sorting of Arrays

- **Sorting of arrays is an arranging elements of array in order .**
- **We need to sort the given array in ascending order such that elements will be arranged from smallest to largest or largest to smallest.**

```
class ArraySort
{
    public static void main(String args[])
{
        // Initial array with unsorted elements
        int array[] = {5, 8, 9, 2, 4, 1, 7, 6};
        System.out.println("Before Sorting: array = ");
        display(array); // Display the array before sorting
```

```
    int temp; // Temporary variable for swapping
    for (int i = 0; i < array.length; i++)
    {
        for (int j = array.length - 1; j > i; j--)
        {
            // Swap elements if they are in the wrong order
            if (array[j - 1] > array[j])
              {
                 temp = array[j];
                 array[j] = array[j - 1];
                 array[j - 1] = temp;
            }
        }
    }
    System.out.println("\nAfter Sorting: array1 = ");
    display(array); // Display the array after sorting
}
```

```
// Method to display array elements

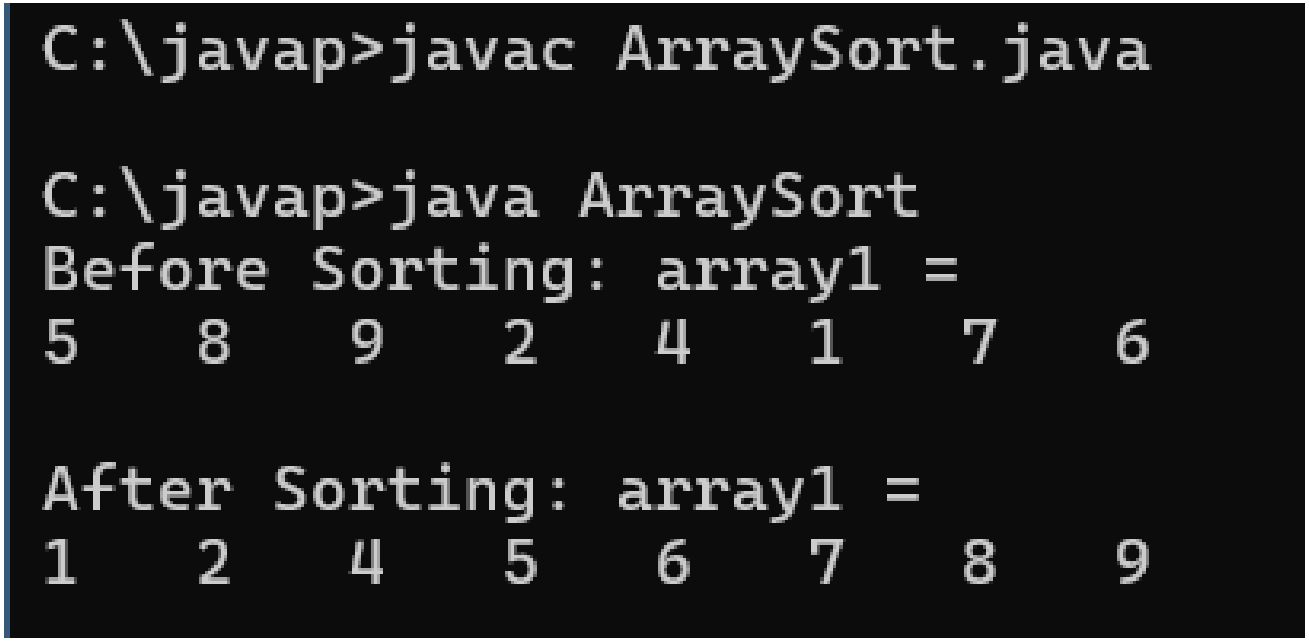
    public static void display(int array[])

{


for(int i=0;i<array.length;i++)
{
System.out.print(array[i]+ "    "); // Print a new line after displaying array

    }

}
```


C:\javap>javac ArraySort.java
C:\javap>java ArraySort
Before Sorting: array1 =
5 8 9 2 4 1 7 6
After Sorting: array1 =
1 2 4 5 6 7 8 9

Unit-III:

Arrays: Introduction, Declaration and Initialization of Arrays, Storage of Array in Computer Memory, Sorting of Arrays, Search for Values in Arrays, Two-dimensional Arrays, Arrays of Varying Lengths, Three-dimensional Arrays.

String Handling in Java: Introduction, Interface Char Sequence, Class String, Methods for Extracting Characters from Strings, Comparison, Modifying, Searching, Class String Buffer, Class String Builder.

PART-B (String Handling in Java)

Concept-1: Introduction and Class String

- ✓ **In Java, string is basically an object that represents sequence of char values. An array of characters works same as Java string.**

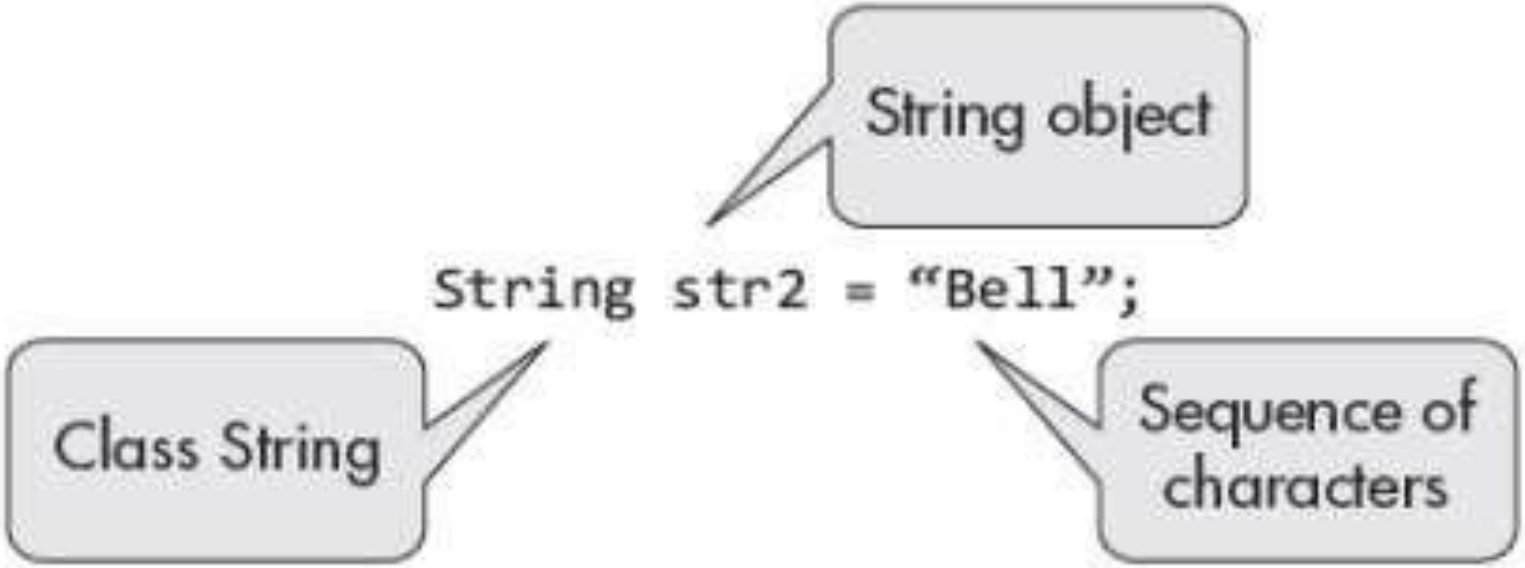
For example:

- ✓ **char ch[ ]={'j','a','v','a'};**
- ✓ **String s=new String(ch);**
- ✓ **String s="java";**

What is String in Java?

- ✓ **Generally, String is a sequence of characters. But in Java, string is an object that represents a sequence of characters. The java.lang.String class is used to create**

a string object.


String object
String str2 = "Bell";
Class String
Sequence of characters

Strings in Java – Declaration, Initialization and Memory Representation

1 Step 1: Declaration

```
String str;
```

str

(reference)

- String is a class in `java.lang` package.
- `str` is a reference variable that can refer to a String object.

Next ➡

2 Step 2: Initialization

```
str = "Hello";
```

str

A String object is created in the String constant pool (SCP) if not already present.

String Constant Pool (SCP)

0	"Hello"

`str` references the String object "Hello" in SCP.

Next ➡

3 Step 3: Another Initialization (New Value)

```
str = "World";
```

str

A new String object "World" is created in the SCP. Now `str` refers to "World".

String Constant Pool (SCP)

	0	1
0	"Hello"	"World"

"Hello" remains in SCP (may be used if referenced).

Next ➡

4 Step 4: String is Immutable

Once a String object is created, its content cannot be changed.

```
str = "Hi";    // creates new object
```

str

String Constant Pool (SCP)

0	1	2
"Hello"	"World"	"Hi"

Now `str` refers to "Hi". Previous objects remain.

Next ➡

5 Step 5: Internal Representation

A String is stored as a sequence of characters (Unicode) terminated by a special character \u0000.

```
String str = "Java";
```

Characters in memory

0	1	2	3	4
'J' (U+004A)	'a' (U+0061)	'v' (U+0076)	'a' (U+0061)	'\u0000' (Null)

Length of "Java" = 4 (Null character is not counted)

Next ➡

6 Step 6: Accessing Characters

Characters are accessed using index (0 to length-1).

```
str = "Java";
str.charAt(0)  // 'J'
str.charAt(1)  // 'a'
str.charAt(2)  // 'v'
str.charAt(3)  // 'a'
```

Index:

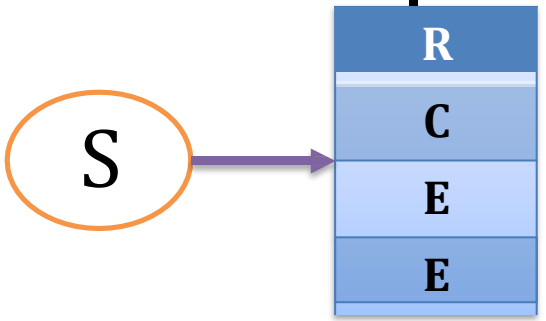
0	1	2	3
J	a	v	a

str

Index starts from 0. Index 4 is invalid (StringIndexOutOfBoundsException).

- Strings are objects of class `java.lang.String` and are stored in the String Constant Pool.
- Strings are immutable and accessed using index. `Length()` returns the number of characters (excluding the null character).

String: String is a group of characters surrounded by double quotes, in java String class is used to represent these group of characters.



Ex: String s="RCEE"

1. Address of the object is assigned to the variable s. This object will be like a character array.

2. Once a string object is created, the data inside the object cannot be modified, that's why we say strings are immutable.

Storage of Strings

- **The memory allocated to a Java program is divided into two segments**

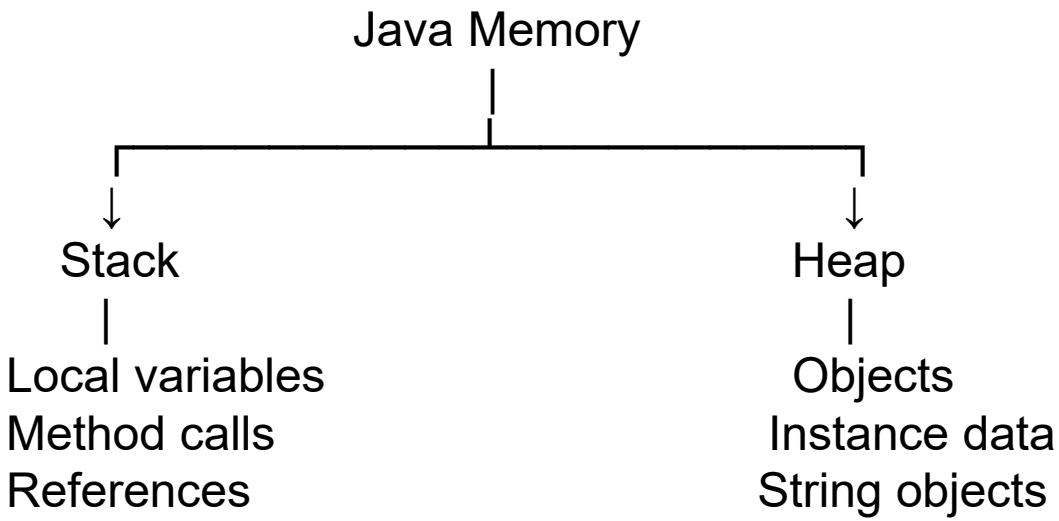
(i) Stack **(ii) Heap**

- **The objects of class String have a special storage facility, which is not available to objects of other two String classes or to objects of any other class.**
- **The variables are stored on heap, whereas the program is stored on stack.**
- **Within the heap, there is a memory segment called 'String constant pool'.**
- **The String class objects can be created in two different ways:**

```
String s = "abcd";

String s = new String("abcd");
```

Immutability :once a String object is created, its content cannot be changed. Any operation that appears to modify a String creates another String object


Java Memory
Stack
Heap
Local variables
Method calls
References
Objects
Instance data
String objects

How to create a string object?

There are two ways to create String object:

1. By string literal

2. By new keyword

1) String Literal

Java String literal is created by using double quotes.

For Example: **String s="welcome";**

Each time you create a string literal, the JVM checks the "string constant pool" first. If the string already exists in the pool, a reference to the pooled instance is returned. If the string doesn't exist in the pool, a new string instance is created and placed in the pool. For example:

1.String s1="Welcome";

2.String s2="Welcome"; **//It doesn't create a new instance**



2) By new keyword

String s=new String("Welcome");

//creates two objects and one reference variable

In such case, JVM will create a new string object in normal (non-pool) heap memory, and the literal "Welcome" will be placed in the string constant pool. The variable s will refer to the object in a heap (non- pool).

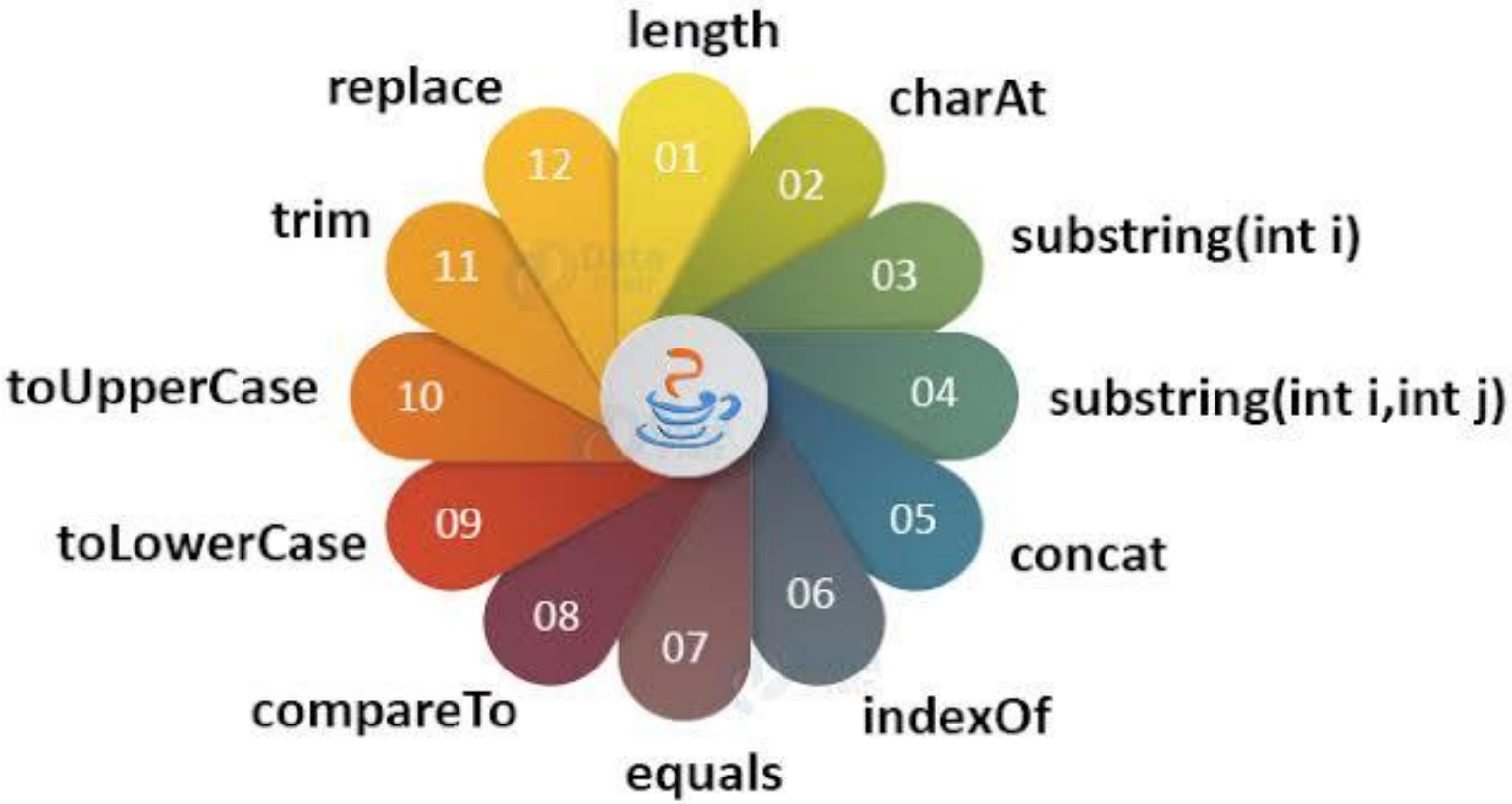
```
import java.lang.*;
class Examp
{
public static void main(String args[])
{
String s1="RCEE";
System.out.println(s1);
String s2=new String("ELURU");
System.out.println(s2);
char ch[]={'E','N','G','I','N','E','E','R'};
String s3=new String(ch);
System.out.println(s3);
String s4="Computer";
System.out.println(s4);
}
}
```

PART-B

Concept-2:Methods for Extracting Characters from Strings

1. charAt()
2. getChars()
3. getBytes()
4. toCharArray()

String Methods in Java



1.charAt():

It is used to extract a single character at an index

Syntax:

```
char charAt(int index)
```

Example:

```
public class CharAtExample
{
    public static void main(String[] args)
 {
        String str = "WELCOME TO AIML";
        char ch = str.charAt(5);
        System.out.println("Character at index 5: " + ch);
    }
}
```

```
C:\javap>javac CharAtExample.java

C:\javap>java CharAtExample
Character at index 5: M
```

2. getChars( )

The getChars() method It is used to extract more than one character

Syntax:

```
public void getChars(int stringBegin, int stringEnd, char dst[], int dstBegin)
```

- ✓ **stringBegin: The starting index (inclusive).**
- ✓ **stringEnd: The ending index (exclusive).**
- ✓ **dst: The destination array.**
- ✓ **dstBegin: The starting index in the destination array.**

Example:

```
public class GetCharsExample
{

    public static void main(String[] args)
{
        String str = "WELCOME TO AIML";
        char dst[] = new char[10];

        str.getChars(1, 5, dst,3);

        System.out.println("Extracted characters: " + new String(dst));
    }
}
```

```
C:\javap>javac GetCharsExample.java

C:\javap>java GetCharsExample
Extracted characters: ELCO
```

3. getBytes()

It is used extract characters from String object and then convert the characters in a byte array

Syntax:

byte getBytes()

Example:

```
public class GetBytesExample
{
    public static void main(String args[])
{
        String str = "Welcome to AIML";

        byte Array[] = str.getBytes();

        for(int i=0;i<str.length();i++)
{

         System.out.print(Array[i]+" ");
    }
}
}
```

```
C:\javap>javac GetBytesExample.java

C:\javap>java GetBytesExample
87 101 108 99 111 109 101 32 116 111 32 65 73 77 76
```

4. toCharArray()

It is used to convert all the characters in a String object into an array of characters.

Syntax:

```
public char[] toCharArray()
```

Example:

```
public class ToCharArrayExample
{
    public static void main(String args[])
{
        String str = "Welcome to AIML";
        char Array[]= str.toCharArray();

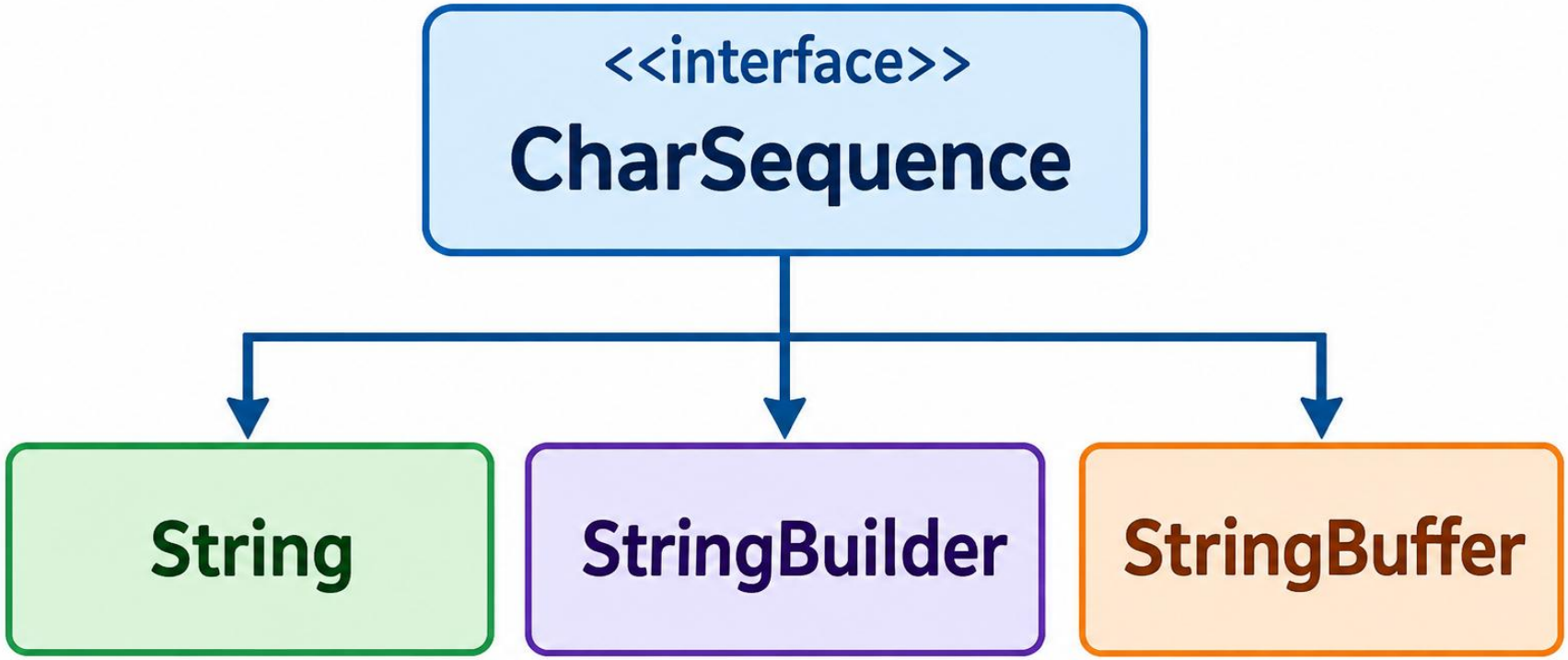
         for(int i=0;i<str.length();i++)
{
         System.out.print(Array[i]+" ");
    }
}
}
```

```
C:\javap>javac ToCharArrayExample.java

C:\javap>java ToCharArrayExample
W e l c o m e   t o   A I M L
```

PART-B

Conept-3:Interface Char Sequence


<<interface>>
CharSequence
String
StringBuilder
StringBuffer

In Java, CharSequence is an interface that represents a sequence of characters. It provides a unified view of various character sequence classes like String, StringBuffer, and StringBuilder. Any class implementing CharSequence can be used to represent text in a flexible way.

Here are some important details about CharSequence:

Methods in CharSequence:

1. length(): Returns the length of the character sequence.

2. charAt(int index): Returns the character at a specific index.

3. subSequence(int start, int end): Returns a new CharSequence that is a subsequence of this sequence, from the start index to the end index.

4. toString(): Returns a String object containing the same character sequence as this

```
class InterfaceCharSequence
{
public static void main(String args[])
{
CharSequence cs="WELCOME , HELLO";
System.out.println("LENGHT IS:"+cs.length());
System.out.println("cHAR AT INDEX IS:"+cs.charAt(2));
System.out.println("SUB SEQUENCE IS:"+cs.subSequence(0,3));
System.out.println("CHAR SEQUENCE TO STRING IS:"+cs.toString());
}
}
```

```
C:\javap>javac InterfaceCharSequence.java

C:\javap>java InterfaceCharSequence
LENGHT IS:15
cHAR AT INDEX IS:L
SUB SEQUENCE IS:WEL
CHAR SEQUENCE TO STRING IS:WELCOME , HELLO
```

Explanation:

- **String class implements CharSequence interface, so it can be treated as a CharSequence.**
- **The flexibility of using CharSequence is that you can work with multiple text representations without being tied to a specific implementation, like String or StringBuilder.**
- **This is useful when working with APIs(application programming interface) that can operate on any character sequences rather than just strings.**

```
class CharseqExample
{
public static void main(String args[])
{
CharSequence cs="welcome rce";
CharSequence chbuf=new StringBuffer("hello rce");
CharSequence chbul=new StringBuilder("hi rce");
System.out.println(cs);
System.out.println("LENGHT IS:"+cs.length());
System.out.println("cHAR AT INDEX IS:"+cs.charAt(2));
System.out.println("SUB SEQUENCE IS:"+cs.subSequence(0,3));
System.out.println("CHAR SEQUENCE TO STRING IS:"+cs.toString());
System.out.println(chbuf);
System.out.println("LENGHT IS:"+chbuf.length());
System.out.println("cHAR AT INDEX IS:"+chbuf.charAt(2));
System.out.println("SUB SEQUENCE IS:"+chbuf.subSequence(0,3));
System.out.println("CHAR SEQUENCE TO STRING IS:"+chbuf.toString());
System.out.println(chbul);
System.out.println("LENGHT IS:"+chbul.length());
System.out.println("cHAR AT INDEX IS:"+chbul.charAt(2));
System.out.println("SUB SEQUENCE IS:"+chbul.subSequence(0,3));
System.out.println("CHAR SEQUENCE TO STRING IS:"+chbul.toString());
}
}
```

```
C:\javap>javac CharseqExample.java

C:\javap>java CharseqExample
welcome rce
LENGHT IS:11
cHAR AT INDEX IS:l
SUB SEQUENCE IS:wel
CHAR SEQUENCE TO STRING IS:welcome rce
hi rce
LENGHT IS:9
cHAR AT INDEX IS:l
SUB SEQUENCE IS:hel
CHAR SEQUENCE TO STRING IS:hello rce
hello rce
LENGHT IS:6
cHAR AT INDEX IS:
SUB SEQUENCE IS:hi
CHAR SEQUENCE TO STRING IS:hi rce
```

PART-B

Concept-4:Comparison, Modifying and Searching

Methods for Comparison of Strings

We can compare String in Java on the basis of content and reference.

It is used in authentication (by equals() method), sorting (by compareTo() method), reference matching (by == operator) etc.

There are three ways to compare String in Java:

1. **By Using equals() Method**
2. **By Using == Operator**
3. **By compareTo() Method**

1) By Using equals() Method

The String class equals() method compares the original content of the string. It compares values of string for equality. String class provides the following two methods:

```
class StringCompare
{
public static void main(String args[])
{
String s1="RAMACHANDRA";
String s2=new String("RAMACHANDRA");
String s3="ENGINEERING";
String s4="Engineering";
System.out.println(s1.equals(s2));
System.out.println(s3.equals(s4));
}
}
```

```
C:\javap>javac StringCompare.java

C:\javap>java StringCompare
true
false
```

2) By Using == operator

The == operator compares references not values.

```
class StringCompare
{
public static void main(String args[])
{
String s1="RAMACNDRA";
String s2=new String("RAMACHANDRA");
String s3="ENGINEERING";
String s4="ENGINEERING";
System.out.println(s1==s2);
System.out.println(s3==s4); //because both refer to same instance
}
}
```

```
C:\javap>javac StringCompare.java

C:\javap>java StringCompare
false
true
```

3) By Using compareTo() method

The String class compareTo() method compares values lexicographically and returns an integer value that describes if first string is less than, equal to or greater than second string.Suppose s1 and s2 are two String objects.

If:.s1 == s2 : The method returns 0.

- s1 > s2 : The method returns a positive value.
- s1 < s2 : The method returns a negative value.

```
class StringCompare
{
public static void main(String args[])
{
String s1="RAMACHANDRA";
String s2=new String("RAMACHANDRA");
String s3="ENGINEERING";
String s4="Engineering";
System.out.println(s1.compareTo(s2));
System.out.println(s3.compareTo(s4));
}
}
```

```
C:\javap>javac StringCompare.java

C:\javap>java StringCompare
0
-32
```

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	Null	32	20	Space	64	40	@	96	60	`
1	01	Start of heading	33	21	!	65	41	A	97	61	a
2	02	Start of text	34	22	"	66	42	B	98	62	b
3	03	End of text	35	23	#	67	43	C	99	63	c
4	04	End of transmit	36	24	$	68	44	D	100	64	d
5	05	Enquiry	37	25	%	69	45	E	101	65	e
6	06	Acknowledge	38	26	&	70	46	F	102	66	f
7	07	Audible bell	39	27	'	71	47	G	103	67	g
8	08	Backspace	40	28	(	72	48	H	104	68	h
9	09	Horizontal tab	41	29	)	73	49	I	105	69	i
10	0A	Line feed	42	2A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	43	2B	+	75	4B	K	107	6B	k
12	0C	Form feed	44	2C	,	76	4C	L	108	6C	l
13	0D	Carriage return	45	2D	-	77	4D	M	109	6D	m
14	0E	Shift out	46	2E	.	78	4E	N	110	6E	n
15	0F	Shift in	47	2F	/	79	4F	O	111	6F	o
16	10	Data link escape	48	30	0	80	50	P	112	70	p
17	11	Device control 1	49	31	1	81	51	Q	113	71	q
18	12	Device control 2	50	32	2	82	52	R	114	72	r
19	13	Device control 3	51	33	3	83	53	S	115	73	s
20	14	Device control 4	52	34	4	84	54	T	116	74	t
21	15	Neg. acknowledge	53	35	5	85	55	U	117	75	u
22	16	Synchronous idle	54	36	6	86	56	V	118	76	v
23	17	End trans. block	55	37	7	87	57	W	119	77	w
24	18	Cancel	56	38	8	88	58	X	120	78	x
25	19	End of medium	57	39	9	89	59	Y	121	79	y
26	1A	Substitution	58	3A	:	90	5A	Z	122	7A	z
27	1B	Escape	59	3B	;	91	5B	[	123	7B	{
28	1C	File separator	60	3C	<	92	5C	\	124	7C	|
29	1D	Group separator	61	3D	=	93	5D	]	125	7D	}
30	1E	Record separator	62	3E	>	94	5E	^	126	7E	~
31	1F	Unit separator	63	3F	?	95	5F	_	127	7F	□

Methods for Modifying Strings

The following methods are used to modify Strings

1. trim() 2. concat() 3. substring() 4. replace() 5.Changing Case

1.trim()

The trim() method removes whitespace from both ends of the string.

Syntax: **public String trim()**

```
public class TrimExample
{
    public static void main(String args[])
 {
        String str = "      WELCOME TO RCE          ";
        String result = str.trim();
        System.out.println("Trimmed string: '" + result + "'");
    }
}
```

2.concat()

The concat() method concatenates the specified string to the end of the original string.

```
public class ConcatExample
{
    public static void main(String args[])
 {
        String str = "RAMACHANRDA";
         String str2="COLLEGE OF ENGINEERING";
        String result = str.concat(str2);
        System.out.println("concatinate string: '" + result + "'");
    }
}
```

```
C:\javap>javac ConcatExample.java

C:\javap>java ConcatExample
Trimmed string: 'RAMACHANRDACOLLEGE OF ENGINEERING'
```

3. substring()

The substring() method returns a new string that is a substring of the original string. It takes one or two arguments: the start index and optionally the end index. Syntax:

- ✓ **public String substring(int beginIndex)**
- ✓ **public String substring(int beginIndex, int endIndex)**
- ✓ **Example:**

```
public class SubstringExample
{
    public static void main(String args[])
 {
        String str = "WELCOME TO RAMACHANDRA";
        String substr1 = str.substring(7); String
        substr2 = str.substring(0, 5);
        System.out.println("Substring from index 7: " + substr1);
        System.out.println("Substring from index 0 to 5: " + substr2);
    }
}
```

```
C:\javap>javac SubstringExample.java

C:\javap>java SubstringExample
Substring from index 7:  TO RAMACHANDRA
Substring from index 0 to 5: WELCO
```

4. replace()

The replace() method replaces all occurrences of a specified character or substring with a new character or substring.

Syntax: public String replace(char oldChar, char newChar)

```
public class ReplaceExample
{
    public static void main(String args[])
{
        String str = "WELCOME TO RCE";
        String result1 = str.replace('O', '7');
        String result2 = str.replace("RCE", "JAVA");
        System.out.println("Replaced 'o' with '0': " + result1);
        System.out.println("Replaced 'World' with 'Java': " +
result2);
    }
}
```

5.Changing Case

Modifying the case of characters in a string.

```
public class ChangingCaseExample
{
    public static void main(String args[])
 {
        String str = "RAMACHANRDA";
        String str2="engineering";
        String result = str.toLowerCase();
        String result1 = str.toUpperCase();
        System.out.println("Changing to lower Case from string: '" +
result + "'");

        System.out.println("Changing to upper Case from string: '" +
result1+ "'");
    }
}
```

```
C:\javap>javac ChangingCaseExample.java

C:\javap>java ChangingCaseExample
Changing to lower Case from string: 'ramachanrda'
Changing to upper Case from string: 'RAMACHANRDA'
```

Methods for Searching Strings

The following methods are used searching Strings

1. indexOf() 2. lastIndexOf() 3. charAt() 4. contains()

1. indexOf( )

The indexOf() method returns the first index at which a specified value can be found in a string. If the value is not found, it returns -1.

```
public class indexOfExample
{
    public static void main(String args[])
 {
        String str = "Hello World";
        int index = str.indexOf("World");
         int notFound = str.indexOf("Python");
        System.out.println("index of World : " +index);
        System.out.println("index of Python : " +notFound);
    }
}
```

```
C:\javap>javac indexOfExample.java

C:\javap>java indexOfExample
index of World : 6
index of Python : -1
```

2. lastIndexOf()

The lastIndexOf() method returns the last index at which a specified value can be found in a string. If the value is not found, it returns -1.

```
public class lastindexOfExample
{
    public static void main(String args[])
 {
          String str = "Welcome to Ramachandra College of Engineering and
Welcome to Eluru";
         int index = str.lastIndexOf("to");
         int notFound = str.lastIndexOf("java");
        System.out.println("last index of World : " +index);
        System.out.println("last index of java : " +notFound);
    }
}
```

```
C:\javap>javac lastindexOfExample.java

C:\javap>java lastindexOfExample
last index of World : 58
last index of Python : -1
```

3.charAt():

It is used to extract a single character at an index

Syntax:

```
char charAt(int index)
```

Example:

```
public class CharAtExample
{
    public static void main(String[] args)
 {
        String str = "WELCOME TO AIML";
        char ch = str.charAt(5);
        System.out.println("Character at index 5: " + ch);
    }
}
```

```
C:\javap>javac CharAtExample.java

C:\javap>java CharAtExample
Character at index 5: M
```

4. contains()

The contains() method checks if a string contains a specified sequence of characters.

```
public class ContainsExample
{
    public static void main(String args[])
 {
        String str = "Welcome to CLASS";
         boolean exists = str.contains("CLASS");
         boolean notExists = str.contains("Python");
        System.out.println("contains of CLASS: " +exists);
        System.out.println("contains of java : " +notExists);
    }
}
```

PART-B

Concept-5: Class String Buffer

- ✓ **Java StringBuffer class is used to create mutable (modifiable) string.**
- ✓ **The StringBuffer class in java is same as String class except it is mutable i.e. it can be changed.**

What is mutable string

- ✓ **A string that can be modified or changed is known as mutable string.**
- ✓ **StringBuffer and StringBuilder classes are used for creating mutable string.**

Constructors of StringBuffer class/Object Creation using StringBuffer

creates an empty string buffer with the initial capacity of 16.

- ❖ **StringBuffer sb=new StringBuffer();**

creates a string buffer with the specified string.

- ❖ **StringBuffer sb=new StringBuffer("Hello");**

creates an empty string buffer with the specified capacity as length.

- ❖ **StringBuffer sb=new StringBuffer(20);**

Inbuilt Methods in the StringBuffer Class

Method	Description
reverse()	It is used to reverse the string.
capacity()	It is used to return the current capacity.
length()	It is used to return the length of the string i.e. total number of characters.
substring(int beginIndex, int endIndex)	It is used to return the substring from the specified beginIndex and endIndex.
replace(int startIndex, int endIndex, String str)	It is used to replace the string from specified startIndex and endIndex.
delete(int startIndex, int endIndex)	It is used to delete the string from specified startIndex and endIndex.
insert(int offset, String s)	It is used to insert the specified string with this string at the specified position.

```
class SBuffer
{
public static void main(String args[])
{
StringBuffer s1=new StringBuffer("Ramacahandra");
StringBuffer s2=new StringBuffer(" College of Engineering");
StringBuffer s3=new StringBuffer();
System.out.println(s1.append(s2));
System.out.println(s1.delete(1,4));
System.out.println(s2.insert(7,"AIML"));
System.out.println(s3.capacity());
System.out.println(s1.reverse());
}
}
```

```
C:\javap>javac SBuffer.java

C:\javap>java SBuffer
RamacahandraCollege of Engineering
RcahandraCollege of Engineering
CollegeAIML of Engineering
16
gnireenignE fo egelloCardnahacR
```

```
class StringBufferExample
{
    public static void main(String args[ ])
 {
StringBuffer sb = new StringBuffer("Hello World"); // Create StringBuffer
        System.out.println("Original String: " + sb);
        sb.reverse(); //Reverse the string
        System.out.println("Reversed String: " + sb);
       sb.reverse(); // Re-reverse to get back to the original string
        System.out.println("Capacity: " + sb.capacity()); //current capacity of the buffer

        System.out.println("Length: " + sb.length());   //length of the string

        String substring = sb.substring(0, 5);//substring(int beginIndex, int endIndex)
        System.out.println("Substring (0,5): " + substring);
        sb.replace(0, 5, "Hi");   //replace(int startIndex, int endIndex, String str)
        System.out.println("After replace: " + sb);
        sb.delete(2, 5); // delete(int startIndex, int endIndex)
        System.out.println("After delete: " + sb);
        sb.insert(2, "llo"); //insert(int offset, String str)
        System.out.println("After insert: " + sb);
    }
}
```

```
C:\javap>javac StringBufferExample.java

C:\javap>java StringBufferExample
Original String: Hello World
Reversed String: dlroW olleH
Capacity: 27
Length: 11
Substring (0,5): Hello
After replace: Hi World
After delete: Hirld
After insert: Hillorld
```

Part-B

Concept-6:Class String Builder.

- ✓ Java StringBuilder class is used to create mutable (modifiable) String.
- ✓ The Java StringBuilder class is same as StringBuffer class except that it is non-synchronized.

Constructors of StringBuilder class/Object Creation using StringBuilder

It creates an empty String Builder with the initial capacity of 16.

- ❖ **StringBuilder sb=new StringBuilder ();**

It creates a String Builder with the specified string.

- ❖ **StringBuilder sb=new StringBuilder ("Hello");**

It creates an empty String Builder with the specified capacity as length.

- ❖ **StringBuilder sb=new StringBuilder (20);**

Inbuilt Methods in the StringBuilder Class

Method	Description
public StringBuilder reverse()	It is used to reverse the string.
public int capacity()	It is used to return the current capacity.
public char charAt(int index)	It is used to return the character at the specified position.
public int length()	It is used to return the length of the string i.e. total number of characters.
public String substring(int beginIndex)	It is used to return the substring from the specified beginIndex.
public String substring(int beginIndex, int endIndex)	It is used to return the substring from the specified beginIndex and endIndex.
public StringBuilder append(String s)	It is used to append the specified string with this string.
public StringBuilder insert(int offset, String s)	It is used to insert the specified string with this string at the specified position.

```
class SBuilder
{
public static void main(String args[])
{
StringBuilder s1=new StringBuilder("Ramacahandra");
StringBuilder s2=new StringBuilder("College of Engineering");
StringBuilder s3=new StringBuilder();
System.out.println(s1.append(s2));
System.out.println(s1.delete(1,4));
System.out.println(s2.insert(7,"Eluru"));
System.out.println(s3.capacity());
System.out.println(s1.reverse());
}
}
```

```
class StringBuilderExample
{
    public static void main(String[] args)
 {

  StringBuilder sb = new StringBuilder("Hello World"); // Create StringBuilder
  System.out.println("Original String: " + sb); //Reverse the string
  sb.reverse();
  System.out.println("Reversed String: " + sb);
  sb.reverse();//Re-reverse
 System.out.println("Capacity: " + sb.capacity()); //current capacity of the builder
 System.out.println("Character at index 6: " + sb.charAt(6)); //character at a specific index
 System.out.println("Length: " + sb.length()); //length of the string
 String substringFrom = sb.substring(6); //substring(int beginIndex)
  System.out.println("Substring from index 6: " + substringFrom);
  String substringBetween = sb.substring(0, 5); //substring(int beginIndex, int endIndex)
  System.out.println("Substring from index 0 to 5: " + substringBetween);
  sb.append("!!!");  //Append a string to the current string
  System.out.println("After appending '!!!': " + sb);
  sb.insert(5, " Java"); //Insert a string at the specified position
  System.out.println("After inserting ' Java' at index 5: " + sb);
    }
}
```

```
C:\javap>javac StringBuilderExample.java

C:\javap>java StringBuilderExample
Original String: Hello World
Reversed String: dlroW olleH
Capacity: 27
Character at index 6: W
Length: 11
Substring from index 6: World
Substring from index 0 to 5: Hello
After appending '!!!': Hello World!!!
After inserting ' Java' at index 5: Hello Java World!!!
```

Additional Information on Arrays and Strings

1. Introduction to Arrays

- Array is a collection of similar type of elements stored in contiguous memory locations.
- Accessed using index (starting from 0).

Array of 5 integers

10	20	30	40	50
0	1	2	3	4

Index: 0 1 2 3 4

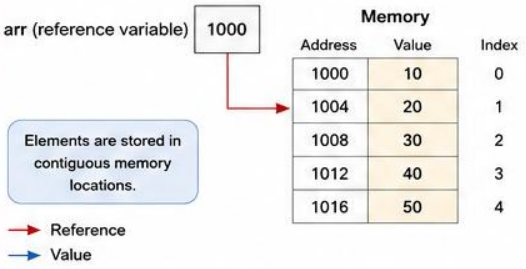
2. Declaration and Initialization

```
int[] arr = new int[5];     // declaration
                            // memory allocation
arr[0] = 10;
arr[1] = 20;
arr[2] = 30;
arr[3] = 40;
arr[4] = 50;
```

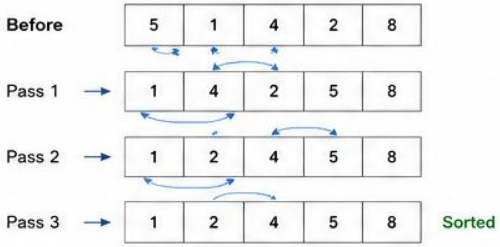
OR

```
int[] arr = {10, 20, 30, 40, 50};
// declaration + initialization
```

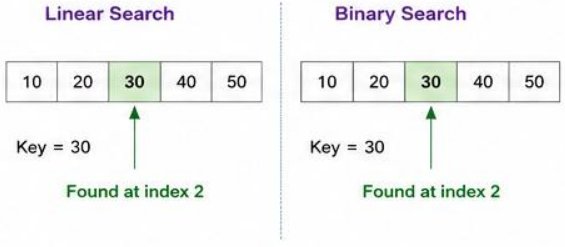
3. Storage of Array in Memory



4. Sorting of Arrays (Bubble Sort)



5. Search for Values in Arrays



6. Two-dimensional Arrays

```
int[][] mat = {{1,2,3}, {4,5,6}, {7,8,9}};
```

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

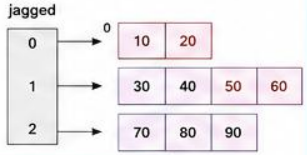
Think of it as a table with rows and columns.

7. Arrays of Varying Lengths (Jagged Array)

```
int[][] jagged = new int[3][];

jagged[0] = new int[2];
jagged[1] = new int[4];
jagged[2] = new int[3];
```

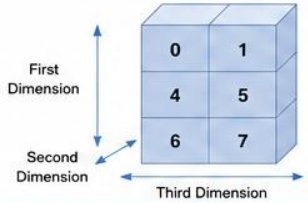
Memory Representation



★ Each row can have different number of columns.

8. Three-dimensional Arrays

```
int[][][] cube = new int[2][2][2];
```



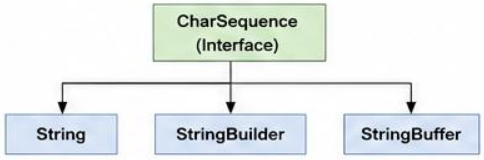
Think of it as an array of 2D arrays (stack of matrices).

9. Introduction to Strings

- String is a sequence of characters.
- In Java, String is an object of java.lang.String class.
- Strings are immutable.

```
String s = "Hello Java";
```

10. Character Sequence Interface



11. String Class (String Constant Pool)

```
String s = "Hello";
```

Memory



(String Constant Pool)

12. Extracting Characters from Strings

```
String s = "Hello";
```

Index:	0	1	2	3	4
	H	e	l	l	o

```
s.charAt(1) → 'e'
s.charAt(4) → 'o'
```

UNIT–III : ARRAYS & STRING HANDLING IN JAVA – CONCEPTS & VISUAL EXPLANATIONS

PART – A : ARRAYS

1 Introduction to Arrays

- Array is a collection of similar type of elements stored in contiguous memory locations.
- Accessed using index (starting from 0).

Array of 5 integers

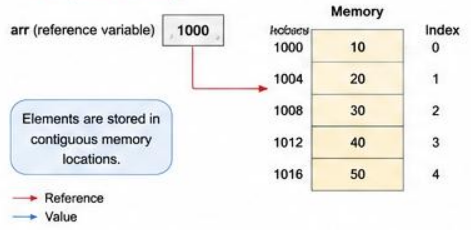
10	20	30	40	50
0	1	2	3	4

Index:

2 Declaration and Initialization

```
int[] arr = new int[5];      // declaration
                             and memory allocation
arr[0] = 10;
arr[1] = 20;
arr[2] = 30;                 OR
arr[3] = 40;
arr[4] = 50;                 int[] arr = {10, 20, 30, 40, 50};
                               // declaration + initialization
```

3 Storage of Array in Computer Memory

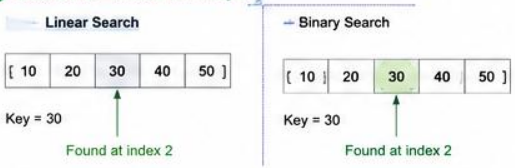


4 Sorting of Arrays

Example: Ascending Order (Bubble Sort)



5 Search for Values in Arrays



6 Two-dimensional Arrays

```
int[][] mat = {{1,2,3}, {4,5,6}, {7,8,9}};
```

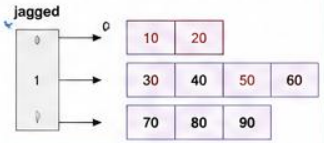
	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

Think of it as a table with rows and columns.

7 Arrays of Varying Lengths (Jagged Array)

```
int[][] jagged = new int[3][];
jagged[0] = new int[2];
jagged[1] = new int[4];
jagged[2] = new int[3];
```

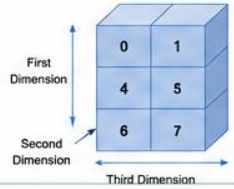
Memory Representation



★ Each row can have different number of columns.

8 Three-dimensional Arrays

```
int[][][] cube = new int[2][2][2];
```



Think of it as an array of 2D arrays (stack of matrices).

TIPS TO REMEMBER

- Index always starts from **0**
- Array size is fixed (for normal arrays)
- Elements are stored in contiguous memory
- Use enhanced for loop for easy traversal

Enhanced For Loop

```
for (int x : arr) {
    System.out.print(x + " ");
}
```

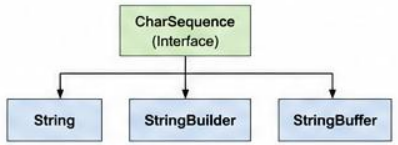
PART – B : STRING HANDLING IN JAVA

1 Introduction to Strings

- String is a sequence of characters.
- In Java, String is an object of *java.lang.String* class.
- Strings are immutable.

```
String s = "Hello Java";
```

2 Character Sequence Interface



3 String Class

```
String s = "Hello";
```

Memory



(String Constant Pool)

4 Extracting Characters from Strings

```
String s = "Hello";
```

Index:	0	1	2	3	4
	H	e	l	l	o

```
s.charAt(1) → 'e'
s.charAt(4) → 'o'
```

5 String Comparison

```
String s1 = "Java";
String s2 = "Java";
String s3 = new String("Java");
```

Method	Result
s1 == s2	true (same reference)
s1 == s3	false (different reference)
s1.equals(s3)	true (same content)
s1.compareTo(s3)	0 (equal)

6 Modifying Strings (Immutable)

```
String s = "Hello";
s = s + " World";    // new String created
```



(Old String is not changed)

7 Searching in Strings

```
String s = "Java Programming";
```

```
s.indexOf('a')     → 1   (first occurrence)
s.lastIndexOf('a') → 12  (last occurrence)
s.contains("Pro")  → true
```

8 StringBuffer Class (Mutable & Synchronized)

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");     // Hello World
sb.insert(6, "Good ");   // Hello Good World
sb.replace(0, 5, "Hi");  // Hi Good World
sb.delete(3, 8);         // Hi World
```

9 StringBuilder Class (Mutable & Non-Synchronized)

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");     // Hello World
sb.reverse();            // dlroW olleH
```

★ Use StringBuilder for performance in single-threaded environment.

WHEN TO USE WHAT?

Use	Why?
String	For fixed/constant text, value should not change (Immutable).
StringBuffer	When many modifications & thread safety is required.
StringBuilder	When many modifications & no thread safety required (faster).

COMMON STRING METHODS

- length() – returns length
- charAt(index) – returns character
- substring(i, j) – returns part of string
- toUpperCase() – converts to upper case
- toLowerCase() – converts to lower case
- trim() – removes spaces

Char array vs String

BASIS FOR COMPARISON	CHARACTER ARRAY	STRING
Basic	Character array is collection of variables, of character data type.	String is class and variables of string are the object of class "string".
Syntax	char array_name [size];	string string_name;
Indexing	An individual character in a character array can be accessed by its index in array.	In string the particular character can be accessed by the function "string_name.charAt(index)".
Data Type	A character array does not define a datatype.	A string defines a datatype in C++.
Operators	Operators in C++ can not be applied on character array.	You may apply standard C++ operator on the string.
Boundary	Array boundaries are easily overrun.	Boundaries will not overrun.

Method	Purpose	Example
append()	Adds data at the end	sb.append("Java")
insert()	Inserts data at a position	sb.insert(2, "Hi")
replace()	Replaces characters	sb.replace(0, 3, "ABC")
delete()	Deletes characters	sb.delete(0, 2)
deleteCharAt()	Deletes one character	sb.deleteCharAt(2)
reverse()	Reverses the sequence	sb.reverse()
charAt()	Returns character at index	sb.charAt(2)
setCharAt()	Changes a character	sb.setCharAt(2, 'X')
substring()	Returns part of the string	sb.substring(1, 4)
subSequence()	Returns character sequence	sb.subSequence(1, 4)
length()	Returns number of characters	sb.length()
capacity()	Returns current capacity	sb.capacity()
ensureCapacity()	Increases capacity if needed	sb.ensureCapacity(50)
setLength()	Changes the length	sb.setLength(10)
indexOf()	Finds first occurrence	sb.indexOf("Java")
lastIndexOf()	Finds last occurrence	sb.lastIndexOf("Java")
toString()	Converts to String	sb.toString()

Thank You..!